

Rust + LSM Storage Engine Stage Plan

Build the project in this order. Each stage introduces only the Rust concepts needed for that part of the engine.

Do not move to the next stage until:

```
cargo test
cargo clippy -- -D warnings
cargo fmt --check
```

all pass.

Stage 0 — Rust foundations and project setup

What you will learn

- Variables and mutability
- Primitive types
- `String`, `str`, `Vec<T>`, and slices
- Structs and enums
- `match`, `if let`, and `while let`
- Functions and methods
- Ownership, moves, borrowing, and references
- `Option<T>` and `Result<T, E>`
- Modules and visibility
- Unit and integration tests
- Cargo commands

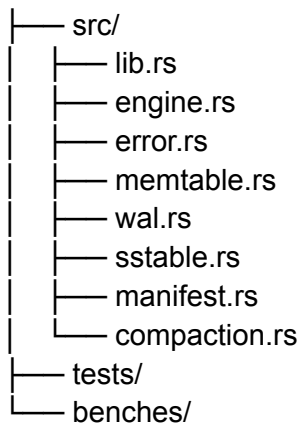
What you must do

Create the project:

```
cargo new lsm-store --lib
cd lsm-store
```

Create this structure:

```
lsm-store/
├── Cargo.toml
```



Complete small Rust exercises involving:

- Moving a `String`
- Borrowing a `String` as `&str`
- Passing a `Vec<u8>` as `&[u8]`
- Returning `Option<T>`
- Propagating an error with `?`
- Matching enum variants
- Writing a struct with an `impl` block

Finished when

You can explain why this takes ownership:

```
fn store(key: Vec<u8>) {}
```

and why this only borrows:

```
fn find(key: &[u8]) {}
```

Phase 1 — In-memory storage

Stage 1 — Build the memtable

The memtable stores the newest writes in memory.

What you will learn

- `BTreeMap`

- Generic collections
- Owned data versus borrowed data
- Mutable and immutable references
- Enums with associated data
- Pattern matching
- Basic test-driven development

What you must build

Create:

```
pub enum Entry {
    Value(Vec<u8>),
    Tombstone,
}
```

Create:

```
pub struct MemTable {
    entries: BTreeMap<Vec<u8>, Entry>,
}
```

Implement:

```
pub fn new() -> Self
pub fn put(&mut self, key: Vec<u8>, value: Vec<u8>)
pub fn get(&self, key: &[u8]) -> Option<&Entry>
pub fn delete(&mut self, key: Vec<u8>)
pub fn contains_key(&self, key: &[u8]) -> bool
pub fn len(&self) -> usize
pub fn is_empty(&self) -> bool
pub fn approximate_size(&self) -> usize
pub fn clear(&mut self)
```

Tests you must write

- Store and retrieve a value
- Missing key returns **None**
- Newest write replaces an older write
- Replacing a value does not increase **len**
- Delete creates a tombstone
- Deleting a missing key creates a tombstone
- Tombstones hide older values
- Approximate size is updated correctly
- Clearing removes everything

Finished when

You understand this complete write:

```
put("name", "Srijan")
```

↓

BTreeMap stores owned key bytes

↓

```
get("name")
```

 borrows the key

↓

returns a borrowed Entry

Stage 2 — Design the public engine API

Create the interface users of the database will call.

What you will learn

- Public versus private modules
- Constructors
- Associated functions
- Custom error types
- `Result`
- Error propagation with `?`
- API design
- Rust documentation comments

What you must build

Create:

```
pub struct Engine {  
    memtable: MemTable,  
}
```

Implement:

```
pub fn open(path: impl AsRef<Path>) -> Result<Self>  
pub fn put(&mut self, key: &[u8], value: &[u8]) -> Result<()>  
pub fn get(&self, key: &[u8]) -> Result<Option<Vec<u8>>>  
pub fn delete(&mut self, key: &[u8]) -> Result<()>  
pub fn flush(&mut self) -> Result<()>  
pub fn close(self) -> Result<()>
```

Create a custom error:

```
pub enum Error {  
    Io(std::io::Error),  
    CorruptedRecord,  
    InvalidChecksum,  
    InvalidManifest,  
    KeyTooLarge,  
    ValueTooLarge,  
}
```

Implement conversions such as:

```
impl From<std::io::Error> for Error
```

Decisions you must document

- Maximum key size
- Maximum value size
- Whether empty keys are allowed
- Whether empty values are allowed
- What deleting a missing key does
- What durability guarantee `put` provides
- Whether `get` returns owned or borrowed bytes

Tests you must write

- Engine can be created
- Put and get work
- Delete works
- Empty values work
- Invalid inputs return errors
- Public API does not expose internal implementation details

Finished when

A user can interact with the database without knowing what a memtable, WAL, or SSTable is.

Phase 2 — Durability

Stage 3 — Define a binary record format

Before writing to disk, define exactly how records are encoded.

What you will learn

- Integer byte representations
- Little-endian and big-endian encoding
- Serialization
- Deserialization
- Buffer construction
- Parsing slices safely
- Avoiding out-of-bounds access

What you must build

Create an operation type:

```
pub enum Operation {  
    Put {  
        key: Vec<u8>,  
        value: Vec<u8>,  
    },  
    Delete {  
        key: Vec<u8>,  
    },  
}
```

Use a record format similar to:

```
checksum    4 bytes  
record type 1 byte  
key length  4 bytes  
value length 4 bytes  
key         variable  
value       variable
```

Implement:

```
pub fn encode(operation: &Operation) -> Result<Vec<u8>>  
pub fn decode(bytes: &[u8]) -> Result<Operation>
```

What you must handle

- Empty input

- Unknown operation type
- Truncated headers
- Truncated keys
- Truncated values
- Invalid lengths
- Oversized records
- Extra bytes
- Invalid checksum

Tests you must write

- Put record round trip
- Delete record round trip
- Empty value round trip
- Unicode bytes round trip
- Binary bytes round trip
- Truncated record fails safely
- Corrupted record fails safely
- Invalid operation type fails safely

Finished when

This works for all valid records:

```
assert_eq!(decode(&encode(&operation)?), operation);
```

Stage 4 — Write-ahead log

The WAL makes writes recoverable after a crash.

What you will learn

- `std::fs::File`
- `OpenOptions`
- Reading, writing, appending, and seeking
- Buffered versus unbuffered I/O
- `write_all`
- `flush`
- Filesystem synchronization
- Resource cleanup through `Drop`
- Partial writes

What you must build

Create:

```
pub struct Wal {  
    file: File,  
    path: PathBuf,  
}
```

Implement:

```
pub fn open(path: impl AsRef<Path>) -> Result<Self>  
pub fn append(&mut self, operation: &Operation) -> Result<u64>  
pub fn sync(&mut self) -> Result<()>  
pub fn replay(&mut self) -> Result<Vec<Operation>>  
pub fn truncate(&mut self) -> Result<()>  
pub fn len(&self) -> Result<u64>
```

Use this write order:

1. Encode operation
2. Append operation to WAL
3. Synchronize WAL
4. Apply operation to memtable
5. Return success

Never update the memtable before the WAL append succeeds.

Tests you must write

- WAL file is created
- Multiple records append correctly
- WAL survives reopening
- Replay preserves record order
- Delete records replay correctly
- Corrupted record is detected
- Truncated final record is handled
- Append failure does not update memtable
- WAL truncate produces an empty log

Finished when

This sequence succeeds:

Open engine

Put A
Put B
Delete A
Terminate engine without a clean shutdown
Open engine again
B exists
A remains deleted

Stage 5 — Crash recovery

Rebuild the memtable by replaying the WAL.

What you will learn

- Iterators
- Stateful parsing
- Lifetimes in parsing code
- Recovery logic
- Idempotence
- Defensive programming
- Failure boundaries

What you must build

When `Engine::open` runs:

1. Open the database directory
2. Open the WAL
3. Read records from oldest to newest
4. Validate every complete record
5. Apply operations to the memtable
6. Ignore or reject an incomplete final record based on your policy
7. Return a usable engine

Implement a WAL iterator:

```
pub struct Walliterator<'a> {  
    // parser state  
}  
  
impl Iterator for Walliterator<'_> {  
    type Item = Result<Operation>;  
}
```

Recovery cases you must test

- Empty WAL
- One valid record
- Hundreds of valid records
- Overwritten keys
- Deleted keys
- Incomplete final header
- Incomplete final value
- Corruption in the middle
- Recovery run multiple times
- WAL containing binary keys and values

Finished when

You can deliberately stop the program after a successful write, reopen the engine, and recover the exact latest state.

Phase 3 — Immutable disk storage

Stage 6 — Flush a memtable into an SSTable

An SSTable is a sorted immutable file.

What you will learn

- Sorted iteration
- File offsets
- File layout design
- Index construction
- Sequential file writing
- Immutable data structures
- Ownership transfer
- Atomic file replacement

What you must design

Use an SSTable format similar to:

[data records]

[index block]
[metadata/footer]

Each data record should contain:

record type
key length
value length
key
value

The index should map keys or key ranges to file offsets.

What you must build

Create:

```
pub struct SsTable {  
    id: u64,  
    path: PathBuf,  
    index: Vec<IndexEntry>,  
}
```

Implement:

```
pub fn create(  
    path: impl AsRef<Path>,  
    id: u64,  
    entries: impl Iterator<Item = (Vec<u8>, Entry)>,  
) -> Result<Self>  
  
pub fn open(path: impl AsRef<Path>) -> Result<Self>  
pub fn get(&self, key: &[u8]) -> Result<Option<Entry>>  
pub fn iter(&self) -> Result<SsTableIterator>  
pub fn smallest_key(&self) -> &[u8]  
pub fn largest_key(&self) -> &[u8]  
pub fn size_bytes(&self) -> u64
```

Flush process

1. Freeze current memtable
2. Create a temporary SSTable file
3. Write sorted entries
4. Write index
5. Write metadata
6. Synchronize the file

7. Rename temporary file to final file
8. Register SSTable
9. Rotate or clear WAL
10. Create a fresh memtable

Tests you must write

- Entries are written in sorted order
- SSTable can be reopened
- Existing key can be found
- Missing key returns **None**
- Tombstone survives flush
- Smallest and largest keys are correct
- Empty table behavior is defined
- Truncated SSTable is rejected
- Invalid footer is rejected
- Temporary file is not treated as a completed SSTable

Finished when

You can write thousands of keys, flush, restart the engine, and read them without replaying the old WAL.

Stage 7 — Build the complete read path

Reads must search data from newest to oldest.

What you will learn

- Layered system design
- Search precedence
- Binary search
- Iterators over multiple sources
- Returning owned data safely
- Early returns
- Tombstone semantics

Read order

1. Mutable memtable
2. Immutable memtable, if one exists
3. Newest SSTable

4. Next-newest SSTable
5. Oldest SSTable

What you must build

Implement:

```
pub fn get(&self, key: &[u8]) -> Result<Option<Vec<u8>>>
```

Rules:

Value found → return value

Tombstone found → return None immediately

Not found → continue to older layer

No layer has key → return None

Add key-range filtering:

If key < SSTable smallest key:
skip table

If key > SSTable largest key:
skip table

Use binary search inside the SSTable index.

Tests you must write

- Memtable value overrides SSTable value
- Newer SSTable overrides older SSTable
- Memtable tombstone hides all older values
- Newer SSTable tombstone hides older values
- Missing key checks all relevant layers
- Tables outside the key range are skipped
- Latest write always wins after multiple flushes
- Reopening preserves search order

Finished when

After arbitrary puts, deletes, flushes, and restarts, **get** always returns the newest logical value.

Phase 4 — Metadata and crash consistency

Stage 8 — Build the manifest

The manifest records which SSTables are part of the database.

What you will learn

- Persistent metadata
- State-machine logs
- File naming
- Monotonically increasing IDs
- Atomic updates
- Directory synchronization
- Crash-consistent file operations

What you must store

The manifest should track:

Next SSTable ID

Active SSTables

SSTable ordering

Compaction state

Storage format version

Possible manifest records:

```
pub enum ManifestRecord {
    AddTable { id: u64 },
    RemoveTable { id: u64 },
    SetNextTableId { id: u64 },
}
```

What you must build

Implement:

```
pub struct Manifest {
    // state
}
```

```
pub fn open(directory: impl AsRef<Path>) -> Result<Self>
pub fn add_table(&mut self, id: u64) -> Result<()>
pub fn remove_tables(&mut self, ids: &[u64]) -> Result<()>
pub fn active_tables(&self) -> &[u64]
pub fn next_table_id(&mut self) -> Result<u64>
pub fn recover(&mut self) -> Result<()>
```

You must define ordering guarantees

For a newly flushed SSTable:

1. Create temporary table
2. Synchronize table
3. Rename to final name
4. Synchronize directory
5. Add table to manifest
6. Synchronize manifest

For compaction:

1. Write new compacted table
2. Synchronize it
3. Rename it
4. Add new table to manifest
5. Remove old tables from manifest
6. Delete obsolete table files

Tests you must write

Simulate failure after every step:

- SSTable written but not renamed
- SSTable renamed but not in manifest
- Manifest updated but old files still exist
- Old table deleted too early
- Temporary files remain
- Manifest final record is truncated
- Unknown file exists in directory
- Missing manifest-referenced file
- Duplicate table IDs

Finished when

After a crash at any tested point, reopening results in either the old valid state or the new valid state—never a half-applied state.

Phase 5 — Compaction

Stage 9 — Implement synchronous compaction

Start with compaction in the calling thread before introducing concurrency.

What you will learn

- Merge algorithms
- Multiple iterators
- Priority queues
- Duplicate resolution
- Tombstone handling
- Streaming transformations
- Avoiding loading entire files into memory

What you must build

Merge multiple sorted SSTables into one.

Example:

Table 3: apple=red, dog=tombstone

Table 2: apple=green, cat=black, dog=brown

Table 1: apple=yellow, bird=blue

Compacted output:

apple=red
bird=blue
cat=black
dog=tombstone

Implement:

```
pub fn compact(  
    input_tables: &[SsTable],  
    output_path: &Path,
```

```
output_id: u64,  
) -> Result<SsTable>
```

Compaction rules

- Newest version of a key wins
- Older versions are removed
- Output remains sorted
- Avoid duplicate keys
- Do not load every table fully into memory
- Preserve tombstones unless it is safe to remove them
- Do not delete source files until the new table is committed

Tests you must write

- Two tables merge correctly
- Five tables merge correctly
- Duplicate keys use newest value
- Tombstones hide older values
- Output is sorted
- Each key appears once
- Empty input tables work
- Compaction output can be reopened
- Crash before manifest update leaves old state usable
- Crash after manifest update leaves new state usable

Finished when

Ten overlapping SSTables can become one SSTable without changing any visible key-value result.

Stage 10 — Background compaction

Move compaction to a separate worker.

What you will learn

- `std::thread`
- Message-passing channels
- `Arc<T>`
- `Mutex<T>`

- `RwLock<T>`
- `Send` and `Sync`
- Thread ownership
- Worker shutdown
- Race conditions
- Lock scope
- Deadlock prevention

What you must build

Create messages:

```
pub enum CompactionMessage {
    Compact,
    Shutdown,
}
```

Create a worker:

```
pub struct CompactionWorker {
    sender: Sender<CompactionMessage>,
    handle: Option<JoinHandle<()>>,
}
```

Implement:

```
pub fn start(/* shared state */) -> Self
pub fn request_compaction(&self) -> Result<()>
pub fn shutdown(&mut self) -> Result<()>
```

Required behavior

- Writes continue while old SSTables are compacted
- Reads can still see a consistent table set
- New compacted table becomes visible atomically
- Old files are deleted only when no read needs them
- Worker shuts down cleanly
- Worker errors are returned or recorded
- Locks are never held during long file writes unless necessary

Tests you must write

- Compaction runs in the background
- Reads work during compaction
- Writes work during compaction

- Multiple compaction requests do not corrupt state
- Shutdown waits for the worker
- Worker failure does not crash the process silently
- SSTable ordering remains correct
- No deleted file is accessed by a concurrent read

Finished when

You can continuously write and read while compaction occurs without incorrect results, panics, or corrupted metadata.

Phase 6 — LSM-tree structure

Stage 11 — Add levels

Organize SSTables into levels instead of keeping one flat list.

What you will learn

- LSM-tree organization
- Leveling strategies
- Size amplification
- Read amplification
- Write amplification
- Overlapping versus non-overlapping key ranges
- Compaction selection policies

Initial design

L0:

- New flushed SSTables
- Key ranges may overlap

L1+:

- Larger tables
- Key ranges should not overlap within a level

What you must build

Track:

```
pub struct Level {
    level_number: usize,
    tables: Vec<SsTableMetadata>,
    max_size_bytes: u64,
}
```

Implement:

- Flushes enter L0
- Trigger compaction when L0 has too many tables
- Select overlapping tables in the next level
- Merge them
- Place output into the next level
- Maintain non-overlapping ranges in L1+
- Continue compaction as a level exceeds its size limit

Configuration to support

```
pub struct Options {
    pub memtable_size_bytes: usize,
    pub l0_compaction_trigger: usize,
    pub level_size_multiplier: usize,
    pub max_levels: usize,
}
```

Tests you must write

- Flush creates an L0 table
- L0 threshold triggers compaction
- Overlapping next-level tables are included
- L1 ranges do not overlap
- Newest values survive cross-level compaction
- Tombstones behave correctly across levels
- Restart reconstructs level structure
- Compaction does not lose keys outside the overlap range

Finished when

The engine behaves like a basic leveled LSM tree rather than a list of immutable files.

Stage 12 — Tombstone garbage collection

Safely remove deletion markers.

What you will learn

- Logical versus physical deletion
- Version visibility
- Garbage-collection safety
- Cross-level correctness
- Data resurrection bugs

What you must do

A tombstone can be removed only when you know no older value can exist in any lower level.

You must create and document a rule such as:

Keep tombstones during ordinary compaction.

Drop a tombstone only when compacting through the final level or when all potentially overlapping older tables are included.

Tests you must write

- Tombstone survives when older values may still exist
- Tombstone is removed when all older values are eliminated
- Deleted data never returns after compaction
- Restart does not resurrect deleted data
- Multiple delete and rewrite cycles behave correctly

Finished when

You can prove through tests that removing tombstones never causes an old value to reappear.

Phase 7 — Performance features

Stage 13 — Bloom filters

Use Bloom filters to avoid unnecessary disk reads.

What you will learn

- Hashing

- Bit arrays
- Probability
- False positives
- Generic hash functions
- Bitwise operations
- Serialization of metadata structures

What you must build

Create:

```
pub struct BloomFilter {
    bits: Vec<u8>,
    hash_count: u32,
}
```

Implement:

```
pub fn new(expected_items: usize, false_positive_rate: f64) -> Self
pub fn insert(&mut self, key: &[u8])
pub fn might_contain(&self, key: &[u8]) -> bool
pub fn encode(&self) -> Vec<u8>
pub fn decode(bytes: &[u8]) -> Result<Self>
```

Store one Bloom filter per SSTable.

Read path

Check key range

↓

Check Bloom filter

↓

If definitely absent: skip file

↓

If possibly present: search index and file

Tests you must write

- Inserted keys are never reported absent
- Missing keys are usually reported absent
- Encode and decode preserve behavior
- Empty filter works
- Invalid serialized filter is rejected
- Read path skips tables when filter says absent

Finished when

Negative lookups perform fewer SSTable reads.

Stage 14 — Block-based SSTables and caching

Avoid reading individual records inefficiently.

What you will learn

- Block-oriented storage
- Locality
- Cache design
- Eviction policies
- Memory limits
- Shared ownership with [Arc](#)
- Interior mutability

What you must build

Change the SSTable format into blocks:

Data block 1

Data block 2

Data block 3

Index block

Bloom filter

Footer

Create a block cache:

```
pub struct BlockCache {  
    // map from table/block ID to cached block  
}
```

Start with a basic bounded LRU-style cache.

Implement:

- Block index
- Block lookup
- Block decoding
- Cache hit

- Cache miss
- Eviction
- Maximum cache size

Tests you must write

- Correct block is selected
- Cache returns previously loaded block
- Cache evicts when over capacity
- Different tables do not collide
- Corrupted block is rejected
- Index offsets remain valid
- Reads spanning different blocks work

Finished when

Repeated reads do not repeatedly load the same disk blocks.

Stage 15 — Range scans

Support ordered iteration across the database.

What you will learn

- Iterator traits
- Iterator adapters
- Multiway merge
- Lifetimes
- Lazy evaluation
- Range bounds
- Deduplicating merged streams

What you must build

Support:

```
pub fn scan(
    &self,
    start: Bound<&[u8]>,
    end: Bound<&[u8]>,
) -> Result<EngineIterator>
```

The iterator must merge:

- Mutable memtable
- Immutable memtable
- L0 SSTables
- Lower levels

Rules:

- Return sorted keys
- Return only newest version
- Hide tombstones
- Respect inclusive and exclusive bounds
- Avoid loading all results into memory

Tests you must write

- Full scan
- Inclusive start
- Exclusive start
- Inclusive end
- Exclusive end
- Empty range
- Duplicate keys across layers
- Tombstones are hidden
- Output is sorted
- Iterator handles thousands of entries lazily

Finished when

The engine supports efficient ordered scans without materializing the entire database.

Phase 8 — Benchmarking and optimization

Stage 16 — Build a workload generator

Do not optimize based on guesses.

What you will learn

- Repeatable benchmarks
- Workload design
- Warmup periods
- Throughput
- Latency distributions
- Percentiles
- Random number generation
- Release-mode performance

Workloads you must create

Sequential writes

key-000001

key-000002

key-000003

Random writes

Random keys distributed across the keyspace.

Read-heavy

90% reads

10% writes

Write-heavy

20% reads

80% writes

Negative lookups

Search for keys that do not exist.

Overwrite-heavy

Repeatedly update a smaller group of keys.

Metrics you must collect

- Operations per second
- Bytes per second
- Average latency
- Median latency
- p95 latency

- p99 latency
- Flush duration
- Compaction duration
- WAL size
- Total SSTable size
- Write amplification
- Read amplification
- Number of files read per lookup
- Bloom-filter skip rate
- Cache hit rate

Important rule

Run performance tests with:

```
cargo build --release
cargo test --release
```

Do not benchmark debug builds.

Finished when

You have a repeatable command that produces a benchmark report.

Stage 17 — Criterion benchmarks

Use Criterion for isolated operations.

What you must benchmark

- Memtable put
- Memtable get
- Record encoding
- Record decoding
- WAL append
- SSTable lookup
- Bloom filter lookup
- Cache hit
- Cache miss
- Two-table merge
- Multi-table merge

Create:

```
benches/  
├── memtable.rs  
├── wal.rs  
├── sstable.rs  
└── compaction.rs
```

Run:

```
cargo bench
```

Finished when

You can compare results before and after a change and identify performance regressions.

Stage 18 — Linux profiling

Find where the engine actually spends time.

What you will learn

- CPU profiling
- Flame graphs
- System calls
- Disk I/O analysis
- Lock contention
- Allocation analysis
- Release symbols

What you must investigate

- Time spent encoding records
- Time spent copying bytes
- Time spent hashing
- Time spent in file synchronization
- Time spent searching indexes
- Time spent waiting for locks
- Compaction CPU usage
- Number of file reads per lookup
- Memory allocations per operation

Optimization rule

For every optimization, record:

Before metric

Change made

Why you expected it to help

After metric

Whether it actually helped

Finished when

Every performance claim is backed by recorded benchmark data.

Phase 9 — Reliability and polish

Stage 19 — Fault injection

Test failures deliberately instead of waiting for them.

What you will learn

- Dependency injection
- Mock I/O
- Failure simulation
- Property-based testing
- Invariants
- Reproducible random tests

Failures you must simulate

- WAL append failure
- WAL sync failure
- SSTable write failure
- SSTable sync failure
- Manifest write failure
- Rename failure
- Compaction interruption
- Partial record
- Corrupted checksum

- Missing file
- Duplicate file
- Unexpected process termination

Invariants you must assert

- A successful write is recoverable
- A failed write is not partially visible
- Keys remain sorted
- Latest write wins
- Deleted values never reappear
- Active manifest files exist
- SSTable IDs are unique
- L1+ ranges do not overlap
- Compaction preserves visible state

Finished when

Random sequences of operations followed by restarts always match a simple reference implementation such as a [BTreeMap](#).

Stage 20 — Documentation and final repository

What you will learn

- Technical documentation
- Architecture communication
- Reproducible experiments
- Project presentation
- Engineering tradeoff analysis

What your README must contain

1. Project summary
2. Features
3. Architecture diagram
4. Write path
5. Read path
6. WAL format
7. SSTable format
8. Manifest design
9. Compaction strategy

10. Crash-recovery guarantees
11. Benchmark methodology
12. Benchmark results
13. Known limitations
14. How to run tests
15. How to run benchmarks
16. Future improvements

Final commands

```
cargo fmt --check  
cargo clippy --all-targets --all-features -- -D warnings  
cargo test  
cargo test --release  
cargo bench
```

Finished when

A new developer can clone the repository, understand the architecture, run the tests, and reproduce the benchmarks.

Recommended timeline

Week 1

- Rust fundamentals
- Ownership and borrowing
- Memtable
- Unit tests

Week 2

- Engine API
- Custom errors
- Binary record encoding
- Record parsing

Week 3

- WAL
- File I/O
- WAL replay
- Crash recovery

Week 4

- SSTable format
- Flushes
- SSTable index
- Persistent reads

Week 5

- Manifest
- File lifecycle
- Atomic updates
- Restart correctness

Week 6

- Synchronous compaction
- Merge iterators
- Tombstone handling

Week 7

- Threads
- Channels
- Shared state
- Background compaction

Week 8

- Levels
- Compaction policy
- Cross-level correctness

Week 9

- Bloom filters

- Blocks
- Block cache

Week 10

- Range scans
- Engine iterator
- Multi-source merge

Week 11

- Benchmark harness
- Criterion
- p95 and p99 latency measurements

Week 12

- Profiling
 - Fault injection
 - Documentation
 - Final results
-

Minimum viable version

Complete these before adding advanced features:

1. Memtable
2. Engine API
3. Record encoding
4. WAL
5. Crash recovery
6. SSTable flush
7. SSTable read path
8. Manifest
9. Synchronous compaction
10. Background compaction
11. Benchmarks

Do not add Bloom filters, caches, async networking, transactions, or distributed features before these work correctly.

Rust concepts you will know by the end

Core language

- Ownership
- Borrowing
- References
- Lifetimes
- Moves
- Cloning
- Structs
- Enums
- Traits
- Generics
- Pattern matching

Error handling

- `Option`
- `Result`
- Custom errors
- Error conversion
- `?`
- Recoverable versus unrecoverable errors

Collections and algorithms

- `Vec`
- `BTreeMap`
- `HashMap`
- Binary search
- Merge algorithms
- Priority queues
- Iterators

Systems programming

- Binary file formats
- File offsets
- Durable writes
- Filesystem synchronization
- Atomic renames
- Crash recovery
- Checksums
- Memory-efficient streaming

Concurrency

- Threads
- Channels
- Arc
- Mutex
- RwLock
- Send
- Sync
- Worker lifecycle
- Race-condition prevention

Performance engineering

- Release builds
- Benchmark design
- Throughput
- Latency percentiles
- Profiling
- Flame graphs
- Allocation reduction
- Cache behavior
- Lock contention

What you should not use initially

Avoid these until the synchronous engine works:

- Tokio
- Async Rust
- Networking

- Transactions
- Replication
- Distributed consensus
- Memory-mapped files
- Lock-free structures
- Custom allocators
- Unsafe Rust

Using them too early will make it harder to learn whether a bug comes from Rust, concurrency, storage logic, or your file format.

Final project target

Your engine should eventually support:

```
let mut db = Engine::open("./data")?;
```

```
db.put(b"name", b"Srijan")?;  
db.put(b"language", b"Rust")?;
```

```
assert_eq!(  
    db.get(b"name")?,  
    Some(b"Srijan".to_vec())  
);
```

```
db.delete(b"name")?;
```

```
assert_eq!(db.get(b"name")?, None);
```

```
db.flush()?;  
db.close()?;
```

After reopening:

```
let db = Engine::open("./data")?;
```

```
assert_eq!(db.get(b"name")?, None);  
assert_eq!(  
    db.get(b"language")?,  
    Some(b"Rust".to_vec())  
);
```

Your strongest final proof is not the number of features. It is this:

After random writes, updates, deletes, flushes, compactions, failures, and restarts, the engine always returns the same state as a reference `BTreeMap`.